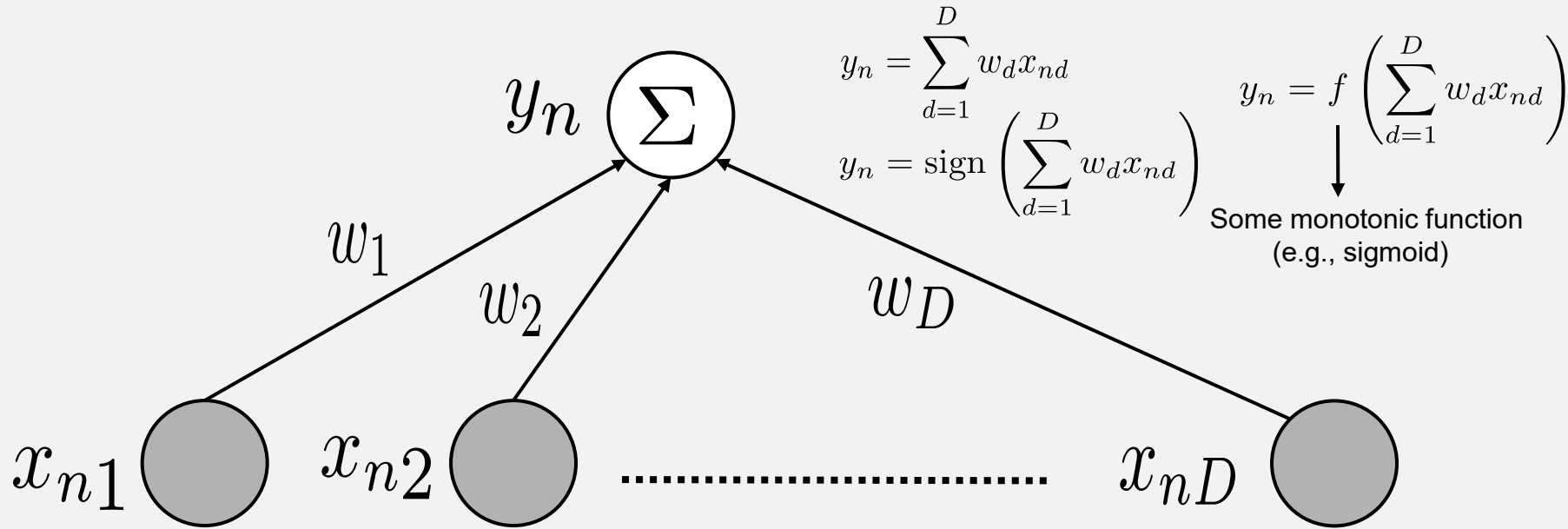


Deep Learning

Perceptron and MLP

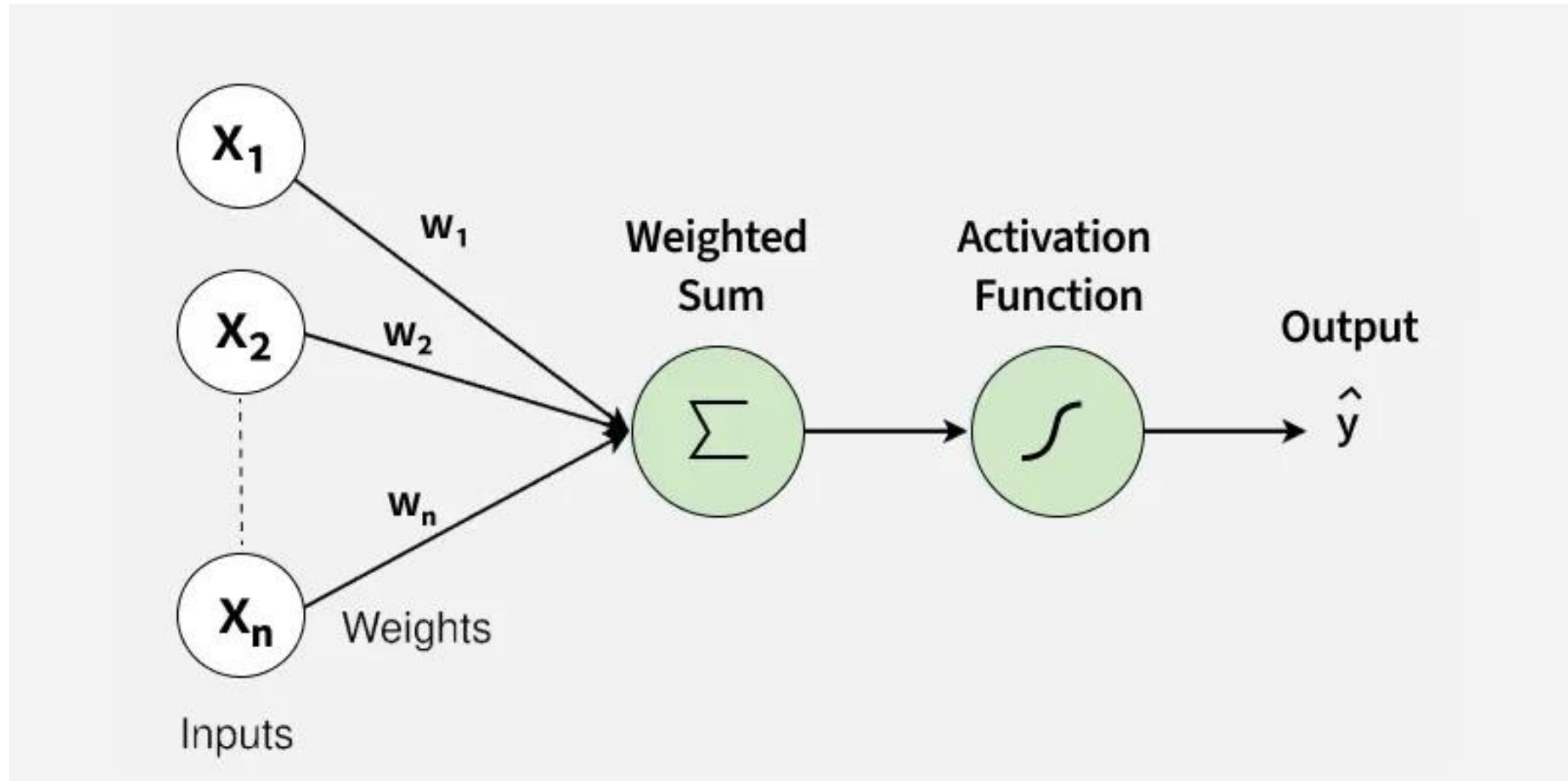
Limitations of Linear Models

- Linear models: Output produced by taking a linear combination of input features



- This basic architecture is classically also known as the “Perceptron” (not to be confused with the Perceptron “algorithm”, which learns a linear classification model)
- This can’t however learn nonlinear functions or nonlinear decision boundaries

Perceptron Architecture



Neural Networks: Multi-layer Perceptron (MLP)

- An MLP consists of an **input layer**, an **output layer**, and **one or more hidden layers**

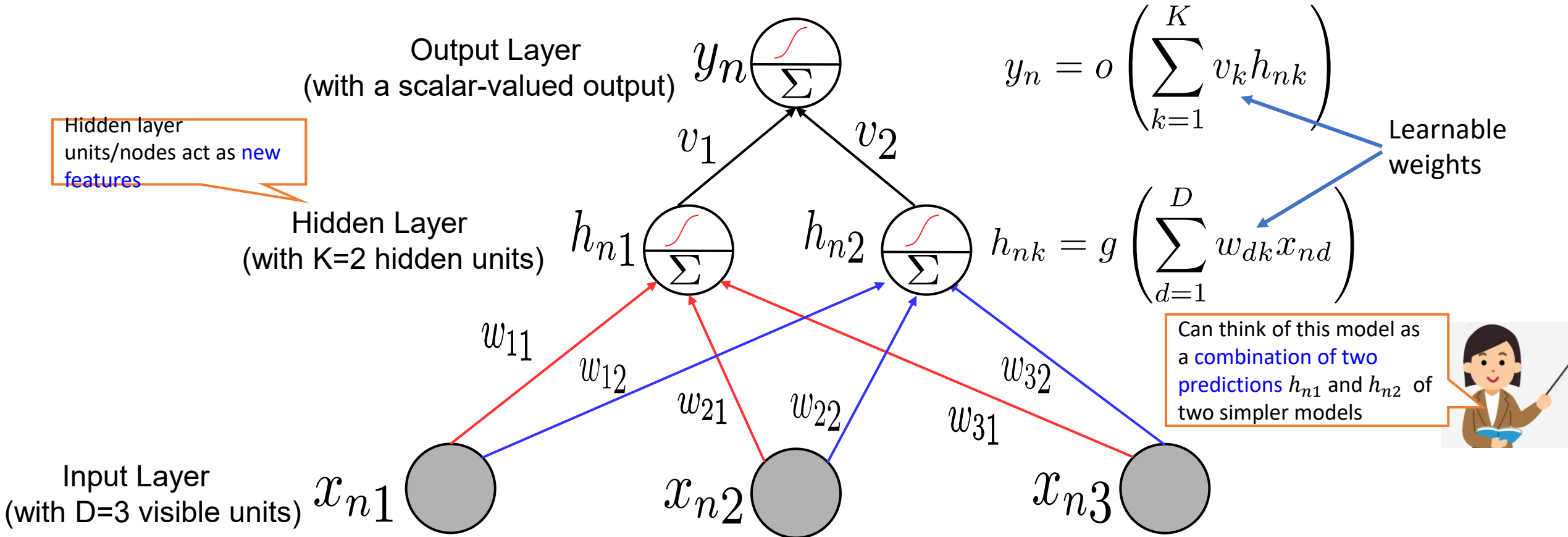


Illustration: Neural Net with One Hidden Layer

- Each input $\mathbf{x}_n$ transformed into several pre-activations using linear models

$$a_{nk} = \mathbf{w}_k^\top \mathbf{x}_n = \sum_{d=1}^D w_{dk} x_{nd}$$

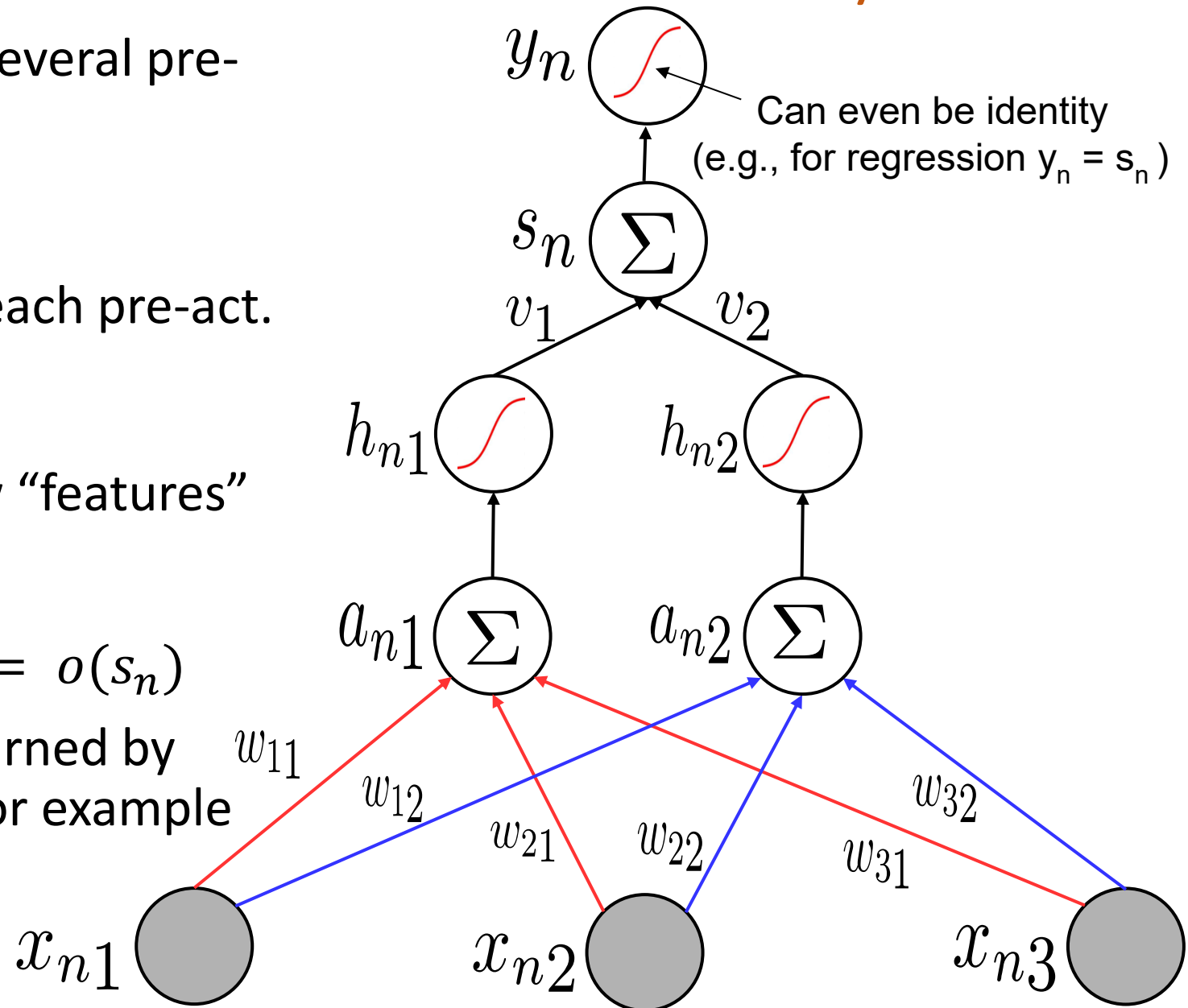
- Nonlinear activation applied on each pre-act.

$$h_{nk} = g(a_{nk})$$

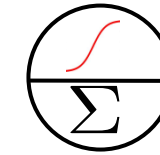
- Linear model learned on the new “features”

$$\mathbf{h}_n \quad s_n = \mathbf{v}^\top \mathbf{h}_n = \sum_{k=1}^K v_k h_{nk}$$

- Finally, output is produced as $y = o(s_n)$
- Unknowns $(\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K, \mathbf{v})$ learned by minimizing some loss function, for example $\mathcal{L}(\mathbf{W}, \mathbf{v}) = \sum_{n=1}^N \ell(y_n, o(s_n))$ (squared, logistic, softmax, etc)



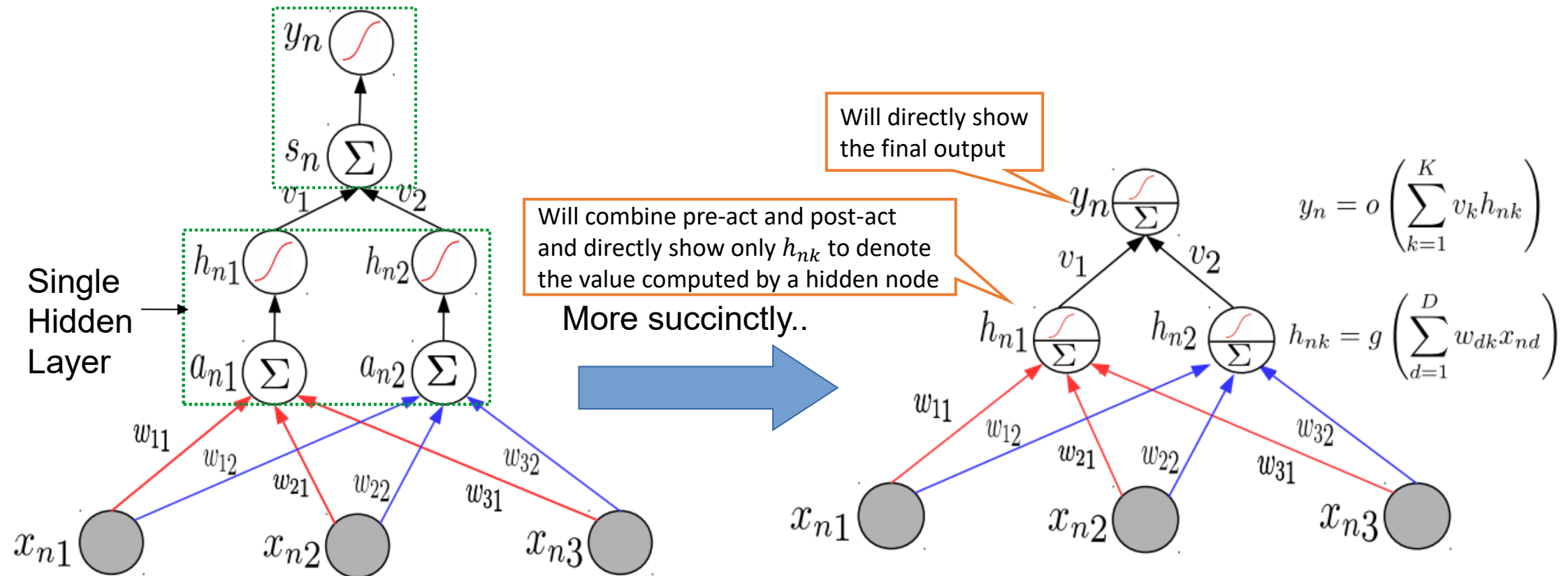
Neural Nets: A Compact Illustration



Will denote a linear combination of inputs followed by a nonlinear operation on the result

6

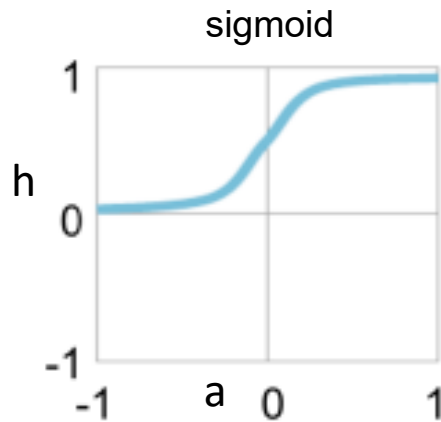
- Note: Hidden layer pre-act a_{nk} and post-act h_{nk} will be shown together for brevity



- Different layers may use different non-linear activations. Output layer may have none.

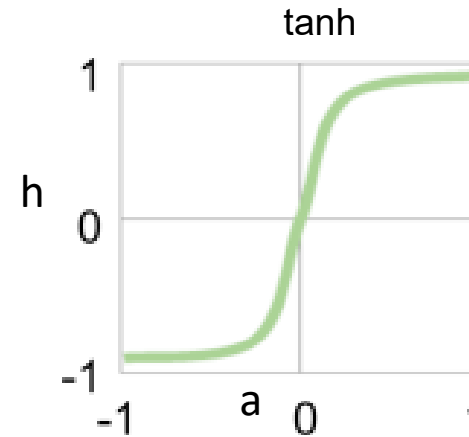
Activation Functions: Some Common Choices

7



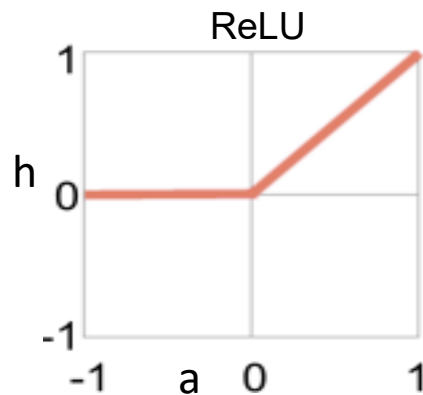
For sigmoid as well as tanh, gradients saturate (become close to zero as the function tends to its extreme values)

Sigmoid: $h = \sigma(a) = \frac{1}{1 + \exp(-a)}$



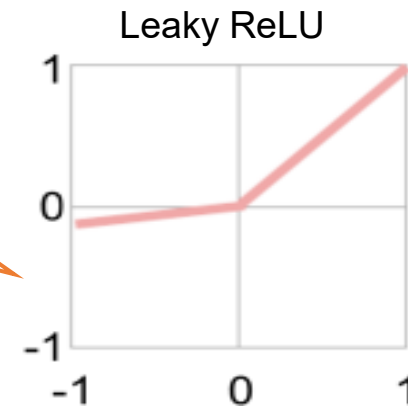
Preferred more than sigmoid. Helps keep the mean of the next layer's inputs close to zero (with sigmoid, it is close to 0.5)

tanh (tan hyperbolic): $h = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)} = 2\sigma(2a) - 1$



Helps fix the dead neuron problem of ReLU when a is a negative number

ReLU (Rectified Linear Unit): $h = \max(0, a)$

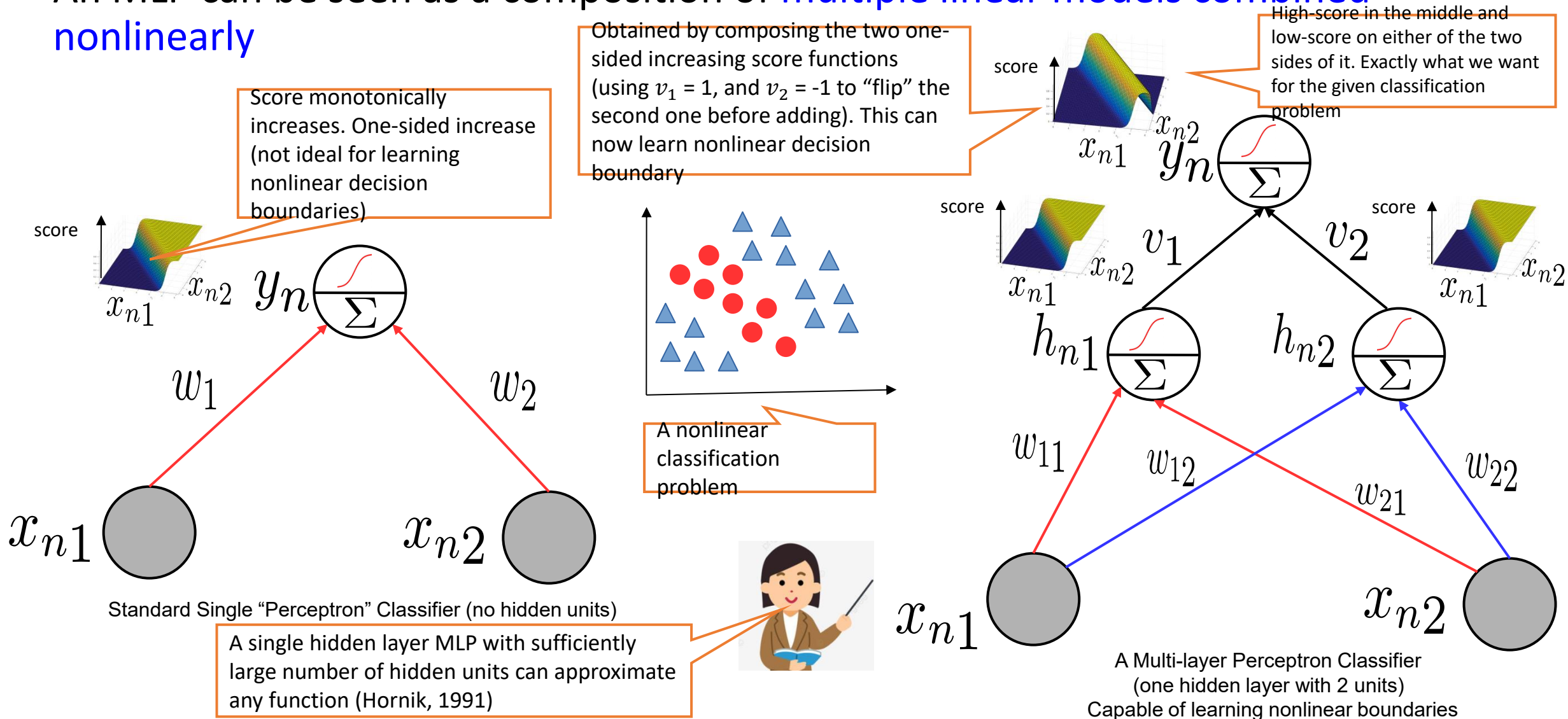


ReLU and Leaky ReLU are among the most popular ones

Leaky ReLU: $h = \max(\beta a, a)$
where β is a small positive number

MLP Can Learn Nonlinear Functions: A Brief Justification⁸

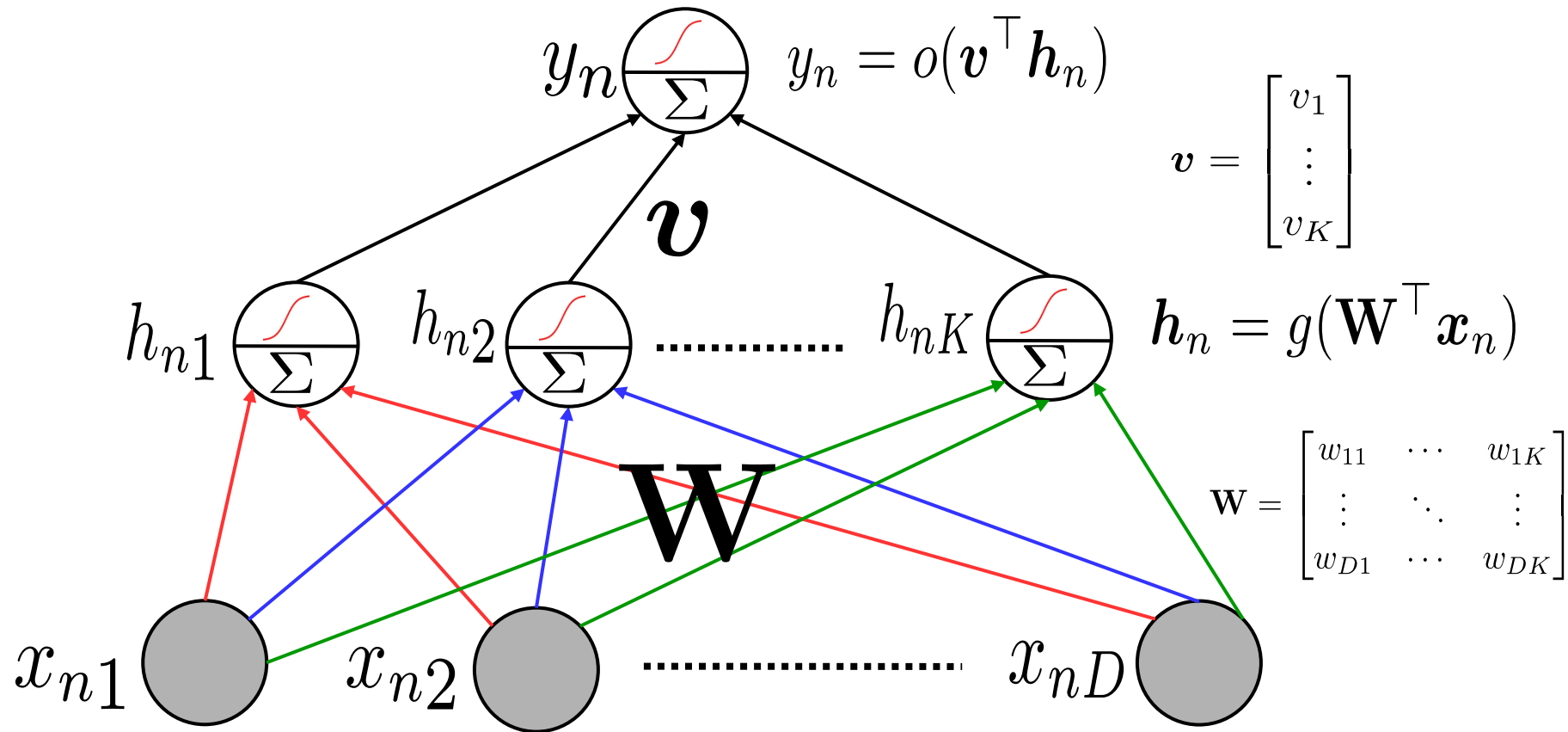
- An MLP can be seen as a composition of **multiple linear models combined nonlinearly**



Examples of some basic NN/MLP architectures

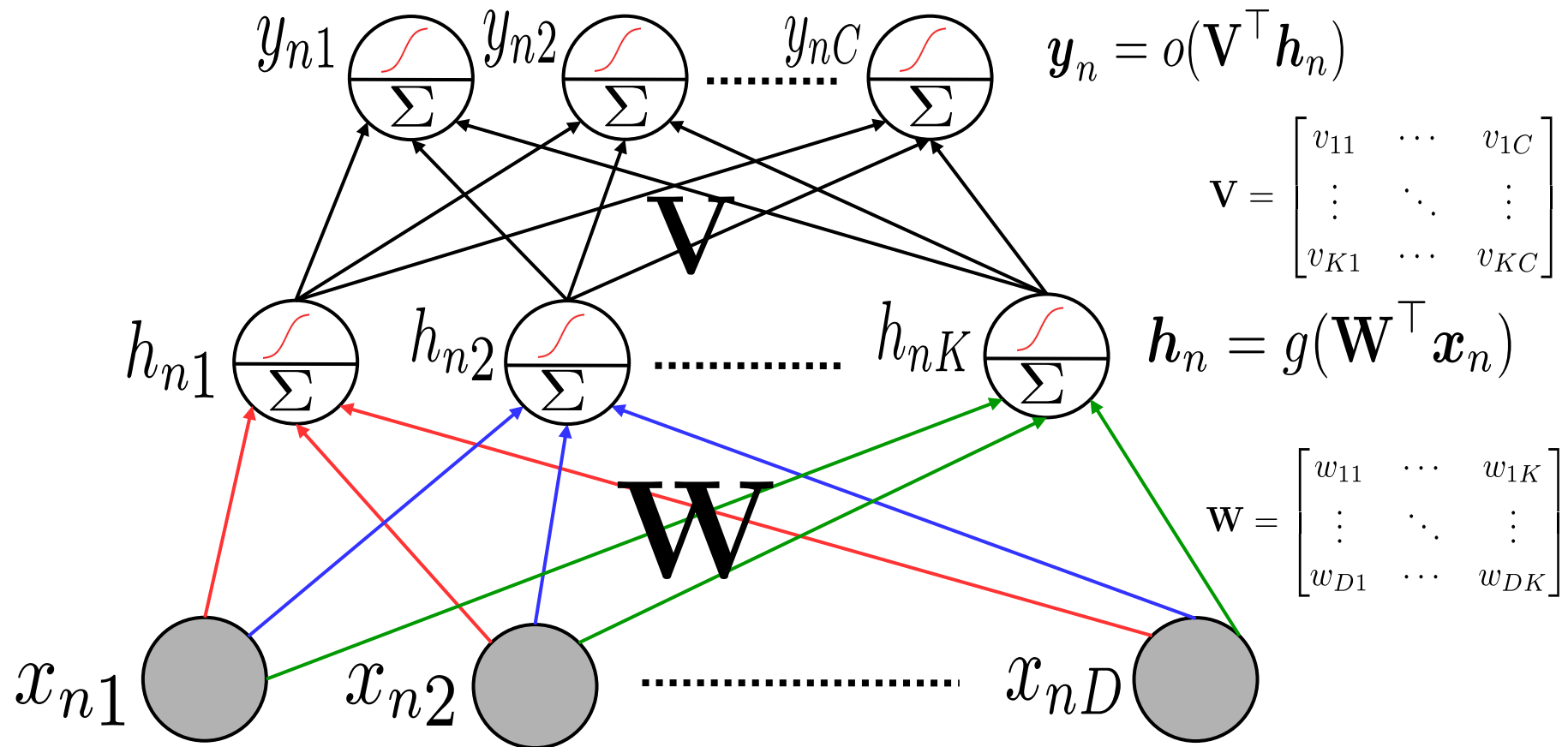
Single Hidden Layer and Single Outputs

- One hidden layer with K nodes and a single output (e.g., scalar-valued regression or binary classification)



Single Hidden Layer and Multiple Outputs

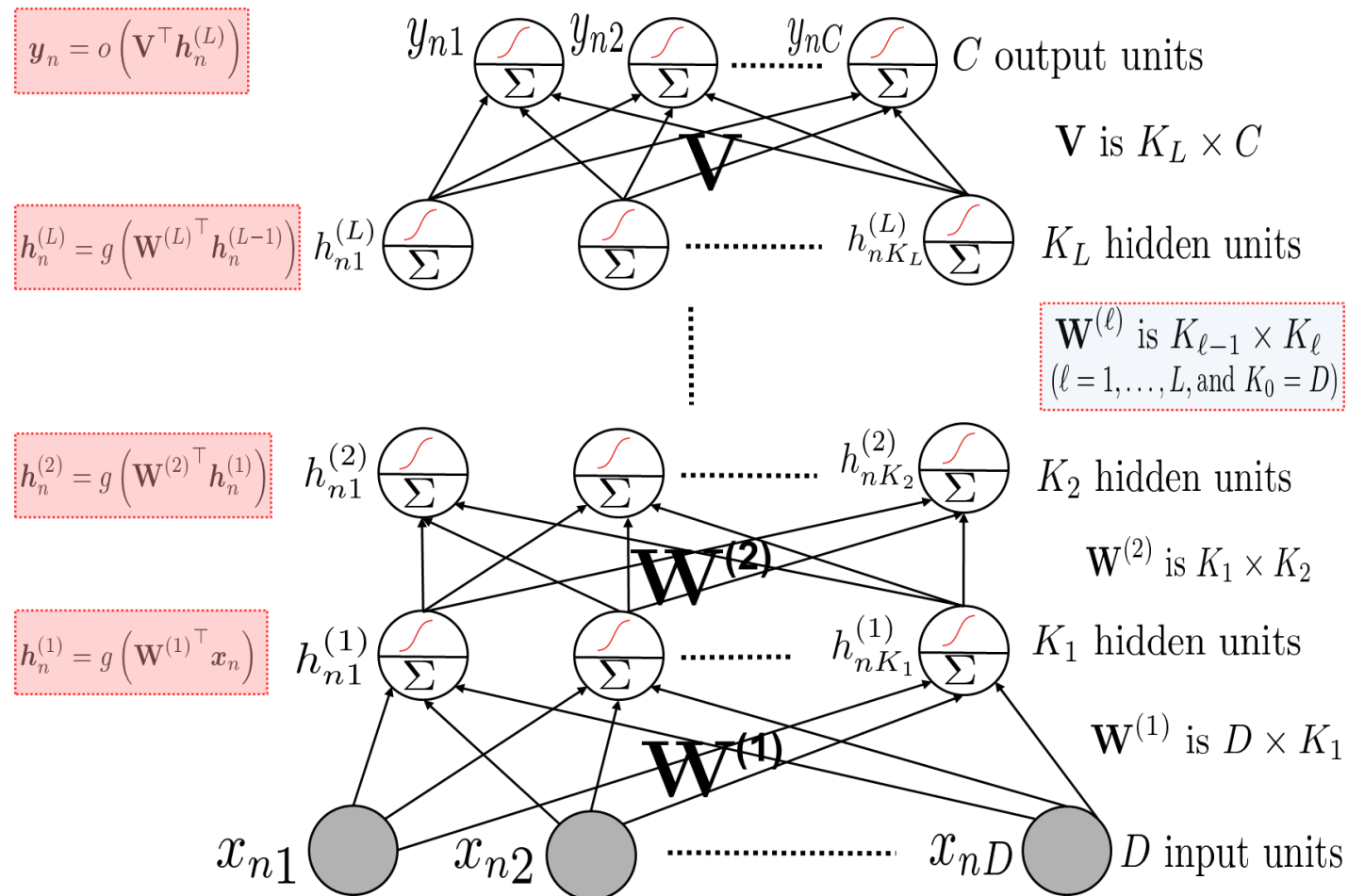
- One hidden layer with K nodes and a vector of C output (e.g., vector-valued regression or multi-class classification or multi-label classification)



Multiple Hidden Layers (One/Multiple Outputs)

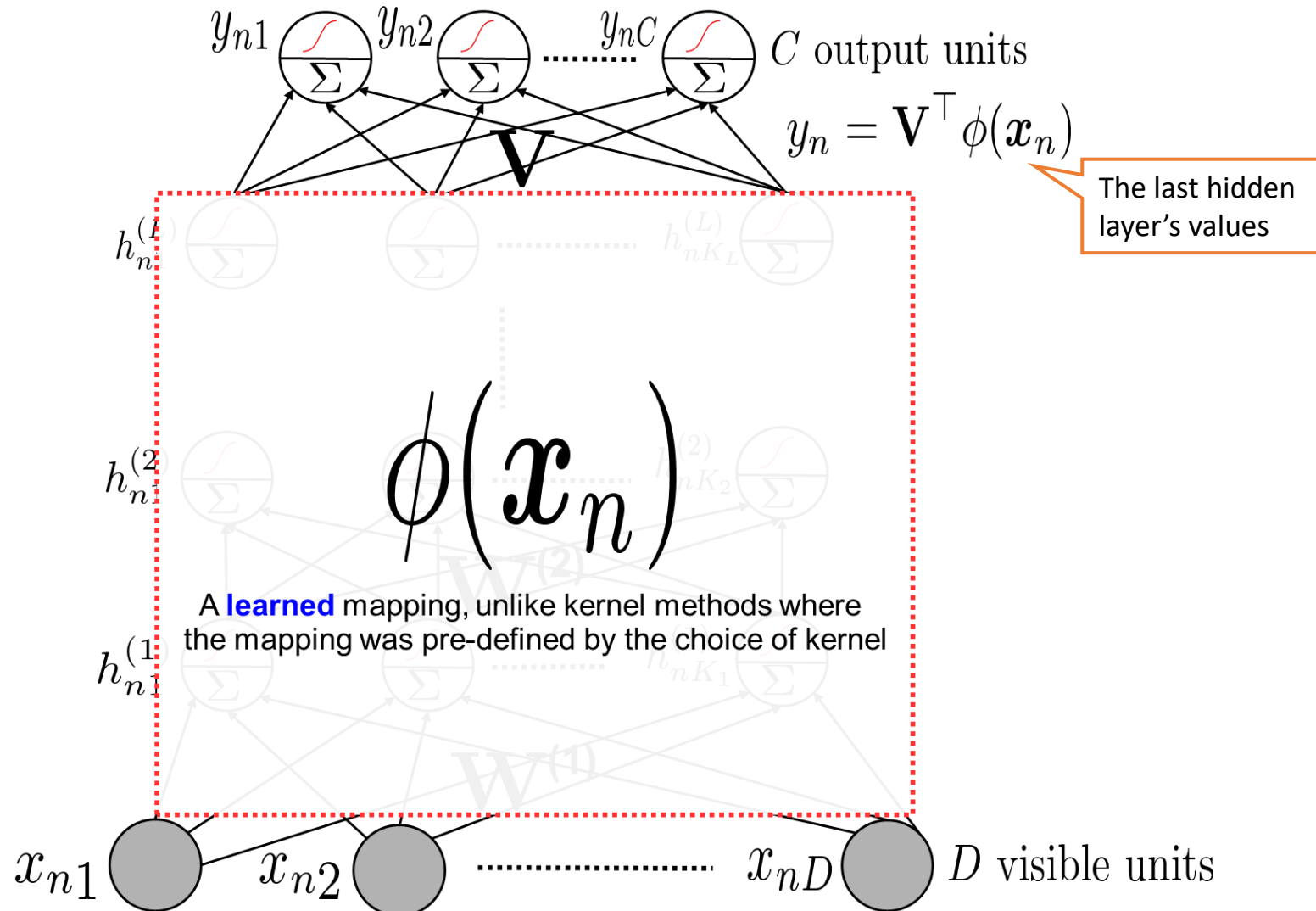
12

- Most general case: Multiple hidden layers with (with same or different number of hidden nodes in each) and a scalar or vector-valued output



Neural Nets are Feature Learners

- Hidden layers can be seen as learning a feature rep. $\phi(\mathbf{x}_n)$ for each input $\mathbf{x}_n$



Kernel Methods vs Neural Nets

- Recall the prediction rule for a kernel method (e.g., kernel SVM)

$$y = \sum_{n=1}^N \alpha_n k(\mathbf{x}_n, \mathbf{x})$$

- This is analogous to a single hidden layer NN with fixed/pre-defined hidden nodes $\{k(\mathbf{x}_n, \mathbf{x})\}_{n=1}^N$ and output weights $\{\alpha_n\}_{n=1}^N$
- The prediction rule for a deep neural network

$$y = \sum_{k=1}^K v_k h_k$$

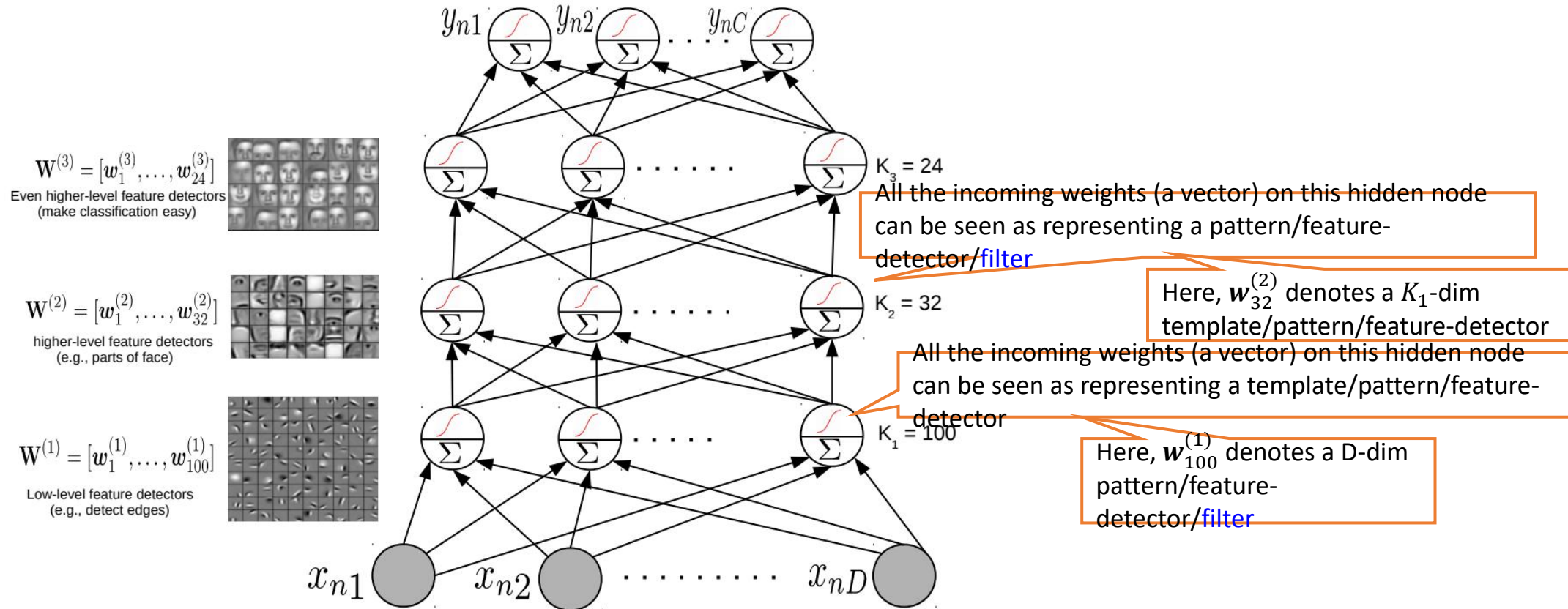
- Here, the h_k 's are learned from data (possibly after multiple layers of nonlinear transformations)
- Both kernel methods and deep NNs be seen as using **nonlinear basis functions** for making predictions. Kernel methods use **fixed basis functions** (defined by the kernel) whereas NN **learns the basis functions adaptively from data**

Also note that neural nets are faster than kernel methods at test time since kernel methods need to store the training examples at test time whereas neural nets do not



Feature Learned by a Neural Network

- Node values in each hidden layer tell us how much a “learned” feature is active in x_n
- Hidden layer weights are like pattern/feature-detector/filter



Why Neural Networks Work Better: Another View¹⁶

- Linear models tend to only learn the “average” pattern
- Deep models can learn multiple patterns (each hidden node can learn one pattern)
 - Thus, deep models can learn to capture more subtle variations that a simpler linear model

